

COMPARATIVA ENTRE KOTLIN MULTIPLATFORM, FLUTTER Y REACT NATIVE

Kotlin Multiplatform (KMP), Flutter y React Native resuelven el desarrollo móvil multiplataforma con estrategias distintas: KMP comparte lógica y conserva UI nativa, Flutter comparte casi toda la app con un motor de render propio, y React Native comparte gran parte del código usando JavaScript y componentes nativos (Janakaraj, 2025; Kumar, 2025). La literatura coincide en que no existe un framework universalmente superior; la elección depende del tipo de proyecto, del conocimiento previo del equipo y de las exigencias de rendimiento, mantenimiento y experiencia nativa (Aripirala et al., 2024; Kumar, 2025; Gupta et al., 2025; Majchrzak & Grønli, 2017).

Dimensión	Kotlin Multiplatform	Flutter	React Native
Lenguaje y curva de aprendizaje	Usa Kotlin; favorece a equipos con base Android/Kotlin. Su tooling y formación siguen madurando (Gupta et al., 2025; Janakaraj, 2025; JetBrains, s.f.).	Usa Dart; suele atraer nuevos desarrolladores, aunque Dart tiene menor base de talento que JS o Kotlin (Gupta et al., 2025; Kumar, 2025).	Usa JavaScript; curva más suave, especialmente para quienes vienen de React/web (Aripirala et al., 2024; Nedyak et al., 2020).
Estrategia de UI	UI nativa por plataforma; comparte lógica sin obligar a una abstracción única de interfaz (Janakaraj, 2025; JetBrains, s.f.).	UI compartida con motor propio de render y widgets; prioriza consistencia visual entre plataformas (Kumar, 2025; Червінська & Афанасьєва, 2025).	UI con componentes nativos controlados desde JavaScript (Oliveira et al., 2023; Червінська & Афанасьєва, 2025).
Qué se comparte	Aproximadamente 70% de la lógica: red, modelos, reglas de negocio; la UI suele implementarse aparte en Android/iOS (Kumar, 2025).	Puede compartir cerca de 90% del código, incluyendo UI y lógica (Kumar, 2025).	También puede compartir cerca de 90% del código, aunque a veces requiere ajustes específicos por plataforma (Kumar, 2025).
Rendimiento y ejecución	Se posiciona cerca de lo nativo al mantener UI nativa y compartir solo capas comunes (Janakaraj, 2025; Jošt & Taneski, 2025).	Compila con AOT a código nativo y suele imponer menos sobrecarga que React Native en tiempo y energía (Jošt & Taneski, 2025; Oliveira et al., 2023).	Ejecuta JavaScript y se comunica con nativo mediante bridge; eso puede introducir cuellos de botella (Oliveira et al., 2023; Kumar, 2025).
Ecosistema, madurez y año de estabilidad	Ecosistema en crecimiento, con librerías en expansión. Alcanzó estabilidad oficial en noviembre de 2023 (Kotlin 1.9.20), por lo que sigue siendo menos maduro que Flutter o RN (Jošt & Taneski, 2025; JetBrains, 2023).	Destaca por tooling integrado, documentación sólida y alta satisfacción de desarrolladores. Alcanzó su primera versión estable (1.0) en diciembre de 2018 (Червінська & Афанасьєва, 2025; Google, 2018).	Comunidad amplia y abundantes librerías, con riesgo de fragmentación de plugins en proyectos grandes. De código abierto desde 2015, con más de una década de iteración continua mediante lanzamientos mensuales (Aripirala et al., 2024; Kumar, 2025).
Empresa respaldante y casos de uso en producción	Impulsado por JetBrains, con respaldo creciente de Google en Android. Usado en producción por Netflix, McDonald's y Philips, entre otras (Jošt & Taneski, 2025; JetBrains, 2023).	Respaldado por Google. Usado en producción por empresas como BMW, Alibaba y Google Ads (Kumar, 2025; Nedyak et al., 2020; Google, 2018).	Respaldado por Meta/Facebook. Usado en producción por Instagram, Shopify, Discord y Bloomberg, entre otras (Kumar, 2025; React Native, s.f.).
Entorno en Windows	Desde Windows puede desarrollarse bien la parte compartida y Android; el trabajo final sobre iOS depende de herramientas del ecosistema Apple por la separación de UI nativa (Kumar, 2025; Janakaraj, 2025).	Puede desarrollarse en Windows, Linux o macOS; la compilación final para iOS igualmente requiere macOS/Xcode (Gülcüoğlu et al., 2021).	También puede desarrollarse en Windows, Linux o macOS; el build final para iOS requiere macOS/Xcode (Gülcüoğlu et al., 2021).

En la evidencia revisada, Flutter y React Native maximizan la reutilización de código, mientras que KMP prioriza experiencia nativa y compartición selectiva de lógica (Kumar, 2025; Janakaraj, 2025). También hay consenso en que los frameworks multiplataforma reducen tiempo y costo frente al desarrollo nativo puro, pero pueden introducir sobrecarga técnica o compromisos de integración según la arquitectura elegida (Biørn-Hansen et al., 2020; Zarichuk, 2023).

Conclusión del equipo

Según lo investigado, elegiríamos Flutter cuando el objetivo principal sea lanzar rápido una app con UI altamente consistente, animaciones y gran porcentaje de código compartido entre plataformas (Kumar, 2025; Червінська & Афанасьєва, 2025). Elegiríamos React Native cuando el equipo ya domina JavaScript/React y necesita acelerar onboarding y prototipado con apoyo de una comunidad amplia, aceptando posibles costos de integración y rendimiento en apps más exigentes (Nedyak et al., 2020; Gülcüoğlu et al., 2021; Kumar, 2025). Elegiríamos Kotlin Multiplatform cuando importe compartir lógica crítica sin perder interfaz nativa, especialmente en equipos que ya trabajan con Android/iOS nativos o en productos donde rendimiento, cumplimiento y experiencia de plataforma pesan más que compartir el 100% de la UI (Kumar, 2025; Janakaraj, 2025; Jošt & Taneski, 2025). Por eso es razonable que el curso use KMP: permite enseñar reutilización real de código y arquitectura multiplataforma sin ocultar cómo funcionan las plataformas nativas ni reemplazar sus APIs e interfaces (Janakaraj, 2025).

Referencias APA 7

- Aripirala, S. S., Airan, P., & Peddinti, D. R. (2024). The Polyglot's Playground: Navigating KMP, Flutter, and React Native in Cross-Platform Ecosystems. *International Journal for Research in Applied Science and Engineering Technology*. <https://doi.org/10.22214/ijraset.2024.64492>
- Biørn-Hansen, A., Rieger, C., Grønli, T.-M., Majchrzak, T. A., & Ghinea, G. (2020). An empirical investigation of performance overhead in cross-platform mobile development frameworks. *Empirical Software Engineering*, 25, 2997–3040. <https://doi.org/10.1007/s10664-020-09827-6>
- Google. (2018, December 4). *Flutter 1.0 launch wrap-up*. The Flutter Blog. <https://flutter.dev/blog/flutter-1-0-launch-wrap-up>
- Gülcüoğlu, E., Ustun, A., & Seyhan, N. (2021). Comparison of Flutter and React Native Platforms. *Journal of Internet Applications and Management*. <https://doi.org/10.34231/iuyd.888243>
- Gupta, N., Gour, A., Raghvendra, N., & Manna, R. (2025). Comparative Analysis of Kotlin, Kotlin Multiplatform, Swift, Flutter, and React Native in Cross-Platform App Development. *International Journal of Innovations in Engineering and Science*. <https://doi.org/10.46335/ijies.2025.10.6.5>
- Janakaraj, M. (2025). Architecting Shared Logic with Kotlin Multiplatform Mobile: Benefits, Challenges, and Ecosystem Positioning. *European Journal of Computer Science and Information Technology*. <https://doi.org/10.37745/ejcsit.2013/vol13n44121141>
- JetBrains. (2023, November 1). *Kotlin Multiplatform is stable and production-ready*. JetBrains Blog. <https://blog.jetbrains.com/kotlin/2023/11/kotlin-multiplatform-stable/>
- JetBrains. (s.f.). *Kotlin Multiplatform documentation*. Kotlinlang. Recuperado en 2026, de <https://kotlinlang.org/docs/multiplatform.html>
- Jošt, G., & Taneski, V. (2025). State-of-the-Art Cross-Platform Mobile Application Development Frameworks: A Comparative Study of Market and Developer Trends. *Informatics*, 12, 45. <https://doi.org/10.3390/informatics12020045>
- Kumar, R. (2025). Evaluating Modern Android Frameworks: A Comparative Study of Flutter, Kotlin Multiplatform, Jetpack Compose, and React Native. *International Journal of Scientific Research in Engineering and Management*. <https://doi.org/10.55041/ijrem48732>
- Majchrzak, T. A., & Grønli, T.-M. (2017). Comprehensive Analysis of Innovative Cross-Platform App Development Frameworks. *Proceedings of the 50th Hawaii International Conference on System Sciences*, 1–10. <https://doi.org/10.24251/hicss.2017.745>
- Nedyak, A., Rudzeyt, O., Zainetdinov, A., & Ragulin, P. (2020). Mobile cross-platform app development tools. *Russian Journal of Resources, Conservation and Recycling*. <https://doi.org/10.15862/13inor420>
- Oliveira, W., Moraes, B., Castor, F., & Fernandes, J. (2023). Analyzing the Resource Usage Overhead of Mobile App Development Frameworks. *Proceedings of the 27th International Conference on Evaluation and Assessment in Software Engineering*. <https://doi.org/10.1145/3593434.3593487>
- React Native. (s.f.). *Showcase archive*. Meta Platforms. Recuperado en 2026, de <https://archive.reactnative.dev/showcase>
- Zarichuk, O. (2023). Comparative analysis of frameworks for mobile application development: Native, hybrid, or cross-platform solutions. *Вісник Черкаського державного технологічного університету*. <https://doi.org/10.62660/2306-4412.4.2023.19-27>
- Червінська, А. Л., & Афанасьєва, І. В. (2025). Mobile cross-platform technologies. Flutter. Xamarin. React Native. *Системи обробки інформації*. <https://doi.org/10.30748/soi.2025.180.12>